Zellic

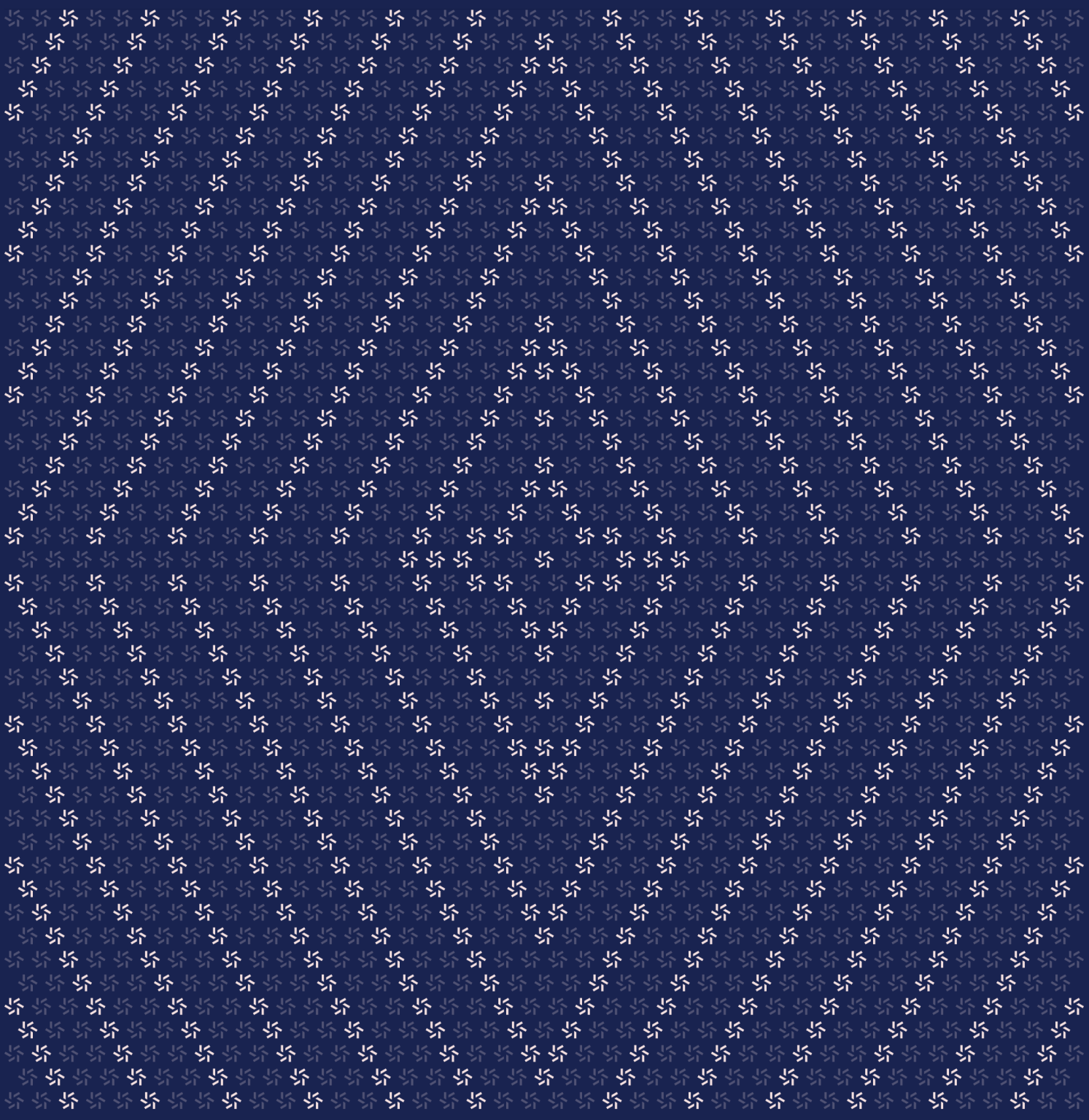
Prepared for
Luka Avbreht
Boštjan Kovač
Flare

Prepared by
Weipeng Lai
Kuilin Li
Zellic

September 29, 2026

Loan Origination - Smart Accounts

Smart Contract Security Assessment



Contents

About Zellic **4**

1. Overview **5**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 6

1.4. Results 6

2. Introduction **7**

2.1. About Loan Origination - Smart Accounts 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 10

2.5. Project Timeline 11

3. Detailed Findings **12**

3.1. A compose sender can incorrectly retain the `ACTION_REDEEM_HOME` executor fee 12

3.2. Fee checkpoints may not credit market interest paid through liquidation 15

3.3. A non-zero fxrpOft bridge fee would block cross-chain moves 17

3.4. Accounts deployed through the singleton factory bypass pausing 19

3.5. ERC-20 approval is not zeroed in some cases 21

4. Patch Review **23**

4.1. Notable changes 23

4.2. Minor changes 24

5. Assessment Results **25**

5.1. Disclaimer 25

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Flare from September 14th to September 24th, 2026. During this engagement, Zellic reviewed Loan Origination - Smart Accounts's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the cross-chain compose authentication sound: can any party other than the account's own Flare twin cause an authorized action on an ownerless account, on either chain?
- Is the replay and retry model correct: can an action execute twice, or a legitimate intent be permanently consumed or permanently stuck, across duplicate deliveries, expiry, reissues, cancellation and the two chains?
- Is the one-permissionless-direction policy actually enforced: is there any path where a caller influences where assets go, or moves assets that are not theirs?
- Is the fee model correct and safe: can it over-charge, under-charge, or leave a user unable to exit, and is the cover gate's arithmetic right at its boundaries and against Morpho's own health check?
- Does the read surface agree with what the actions enforce, and can a quote mislead a user into a transaction that cannot succeed or that costs more than shown?
- Is the Morpho integration correct across the full position lifecycle - collateral supply, borrowing, repayment, external repayment, liquidation and closure - including the oracle and share-conversion arithmetic?
- Is the deterministic address and implementation scheme safe: do the frozen proxy creation codes, the per-type per-chain registry, the initialization path and the storage rule hold across all lineages?
- Is the multi-chain routing model safe: the write-once type-to-chain pin, the suspension lever, the adapter rotation, and the destination EID on the cross-chain user operation?
- Is the access-control and containment design appropriate: is the split between timelocked, immediate and permanent operations right, and can the levers contain the incidents they are meant to contain?
- Is the native and token handling on the new accounts safe: can value strand, be double-spent, or be taken by the wrong party?
- Does the change have any unintended effect on the previously audited in-scope contracts it does not directly modify?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

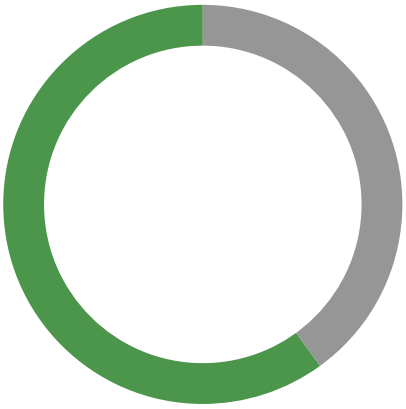
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Loan Origination - Smart Accounts contracts, we discovered five findings. No critical issues were found. Three findings were of low impact and the remaining findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	3
Informational	2



2. Introduction

2.1. About Loan Origination - Smart Accounts

Flare contributed the following description of Loan Origination - Smart Accounts:

> Flare Smart Accounts is an account-abstraction layer that gives every XRPL address a smart account on Flare, controlled only by that address through XRPL payment transactions, so XRPL users can act on Flare without holding FLR. It is a single EIP-2535 diamond (MasterAccountController) with per-user Personal Accounts behind a frozen CREATE2 proxy. This engagement reviews loan origination: a cross-chain lending flow in which a user's XRPL-signed instruction bridges FXRP to a deterministic loan subaccount at the same address on a remote chain (Ethereum at launch), where an authenticated LayerZero compose message supplies it as collateral on a Morpho market, borrows against it and disburses the proceeds. It also adds the accounts that make the remote chain usable for an XRPL-only user, and a Flare fee charged on the loan.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look

for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Loan Origination - Smart Accounts Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only changes between 91244b01...6905db84
Repository	https://github.com/flare-foundation/flare-smart-accounts ↗
Version	6905db84cdc642d59c104584dc939c543d861f18
Programs	smartAccounts/implementation/MorphoLoanSubaccountRemote.sol smartAccounts/implementation/SubaccountBase.sol smartAccounts/implementation/ComposeReceiverAccountBase.sol smartAccounts/implementation/ControlledAccountBase.sol smartAccounts/library/LoanMath.sol smartAccounts/facets/PersonalAccountsFacet.sol smartAccounts/library/PersonalAccounts.sol smartAccounts/implementation/PersonalAccount.sol smartAccounts/implementation/PersonalAccountBase.sol smartAccounts/facets/InstructionsFacet.sol smartAccounts/facets/MemoInstructionsFacet.sol smartAccounts/facets/PauseFacet.sol smartAccounts/facets/ReaderFacet.sol utils/implementation/OwnableWithTimelock.sol smartAccounts/proxy/SubaccountProxy.sol smartAccounts/facets/BridgeRoutesFacet.sol smartAccounts/library/BridgeRoutes.sol smartAccounts/facets/SubaccountsFacet.sol smartAccounts/library/Subaccounts.sol smartAccounts/facets/MorphoLoanMarketsFacet.sol smartAccounts/library/MorphoLoanMarkets.sol smartAccounts/facets/ComposeTransportFacet.sol smartAccounts/library/ComposeTransport.sol smartAccounts/library/MorphoMath.sol smartAccounts/implementation/BridgingSubaccountBase.sol smartAccounts/implementation/LoanSubaccount.sol smartAccounts/implementation/PersonalAccountRemote.sol smartAccounts/facets/EthereumControllerInit.sol

```
smartAccounts/library/FXrp.sol
smartAccounts/library/MemoInstructions.sol
composer/implementation/FAssetRedeemComposer.sol
composer/implementation/FAssetRedeemerAccount.sol
composer/proxy/FAssetRedeemComposerProxy.sol
composer/proxy/FAssetRedeemerAccountProxy.sol
smartAccounts/facets/AgentVaultsFacet.sol
smartAccounts/facets/DiamondCutFacet.sol
smartAccounts/facets/ExecutorsFacet.sol
smartAccounts/facets/FacetBase.sol
smartAccounts/facets/InstructionFeesFacet.sol
smartAccounts/facets/MasterAccountControllerInit.sol
smartAccounts/facets/PaymentProofsFacet.sol
smartAccounts/facets/TimelockFacet.sol
smartAccounts/facets/VaultsFacet.sol
smartAccounts/facets/XrplProviderWalletsFacet.sol
smartAccounts/implementation/MasterAccountController.sol
smartAccounts/library/AgentVaults.sol
smartAccounts/library/Executors.sol
smartAccounts/library/InstructionFees.sol
smartAccounts/library/Instructions.sol
smartAccounts/library/Pause.sol
smartAccounts/library/PaymentProofs.sol
smartAccounts/library/PaymentReferenceParser.sol
smartAccounts/library/Timelock.sol
smartAccounts/library/Vault.sol
smartAccounts/library/Vaults.sol
smartAccounts/library/XrplProviderWallets.sol
```

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.7 person-weeks. The assessment was conducted by two consultants over the course of 9 calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Weipeng Lai
Security Engineer
weipeng.lai@zellic.io ↗

Kuilin Li
Security Engineer
kuilin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 14, 2026	Start of primary review period
September 15, 2026	Kick-off call
September 24, 2026	End of primary review period

3. Detailed Findings

3.1. A compose sender can incorrectly retain the `ACTION_REDEEM_HOME` executor fee

Target	BridgingSubaccountBase		
Category	Coding Mistakes	**Severity**	Low
Likelihood	High	**Impact**	Low

Description

This diff adds the BridgingSubaccountBase contract, which allows the LoanSubaccount to receive LayerZero messages for the `ACTION_REDEEM_HOME` action, which is the last cleanup step when FXRP returns home from the loan chain.

Normally, if the `sweepRedeem` function is called from a non-cross-chain context, the caller is required to include an executor fee that's dependent on the configuration in the controller:

```
function sweepRedeem()
    external payable
    nonReentrant
    returns (uint256 _amount, uint256 _redeemedAmount)
{
    // a public sweep must fund the configured executor fee, including the
    dust branch;
    // this bounds the accepted permissionless-direction griefing
    // slither-disable-next-line unused-return
    (, uint256 executorFee)
    = IExecutorsFacet(controllerAddress).getExecutorInfo();
    require(msg.value >= executorFee,
    InsufficientFundsForRedeem(executorFee));
    return _sweepRedeem(msg.value, type(uint256).max);
}
```

However, no such check exists in the cross-chain context. The `_acceptsComposeValue` function allows compose value to be sent through `lzCompose`, but `_executeRedeemHome` only passes through the value to `_sweepRedeem`:

```
function _executeRedeemHome(
    bytes memory _actionData,
    bytes32 _guid,
    uint256 _amountLD
)
```

```
    private
{
    address parent = _parentAccount();
    uint64 deadline = abi.decode(_actionData, (uint64));
    if (block.timestamp > deadline) {
        // [...]
        return;
    }
    _sweepRedeem(msg.value, _amountLD);
}
```

So, the caller of `lzCompose` can include zero native value without issue.

Impact

In the ideal workflow, the caller of `lzCompose` would be a LayerZero executor who includes the expected amount of native value. The executor is reimbursed out of the fees paid to LayerZero after the execution transaction is verified.

However, the executor can instead decide to set the native value to zero, and keep the fees. LayerZero does not check the value, which means their verification will succeed.

Additionally, because execution is permissionless, anyone who wishes to grief this cross-chain interaction can also front-run and call `lzCompose` with the same parameters that the executor would use, but with zero value. This would cause the real execution to fail because the message would already be consumed, but the subaccount is still left without its native value refund.

Recommendations

We recommend adding a native value check to the `_executeRedeemHome` function.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit `4210cfbb` ↗.

Flare provided the following comment:

> The redeem-home compose now uses the same redemption function as the public sweepRedeem. That function first requires the supplied native value to cover the controller's executor fee (InsufficientFundsForRedeem). An underfunded delivery reverts, and the packet stays queued for a correctly funded retry, so neither the executor nor a front-runner can consume it without supplying the required fee. The deadline branch, which could finalize a packet with any value, is removed. Redeem-home now also redeems the account's whole balance, as the

public sweep does.

3.2. Fee checkpoints may not credit market interest paid through liquidation

Target	MorphoLoanSubaccountRemote		
Category	Business Logic	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

The Flare fee is designed so that the borrower's effective rate is `max(feeFloor, Morpho rate + feeSpread)`, measured between fee checkpoints. At each checkpoint, MorphoLoanSubaccountRemote prices the elapsed period with `LoanMath.feeDeltaX18`. This function infers the market interest for the period from the difference between the live Morpho debt and the principal recorded at the previous checkpoint:

```
function feeDeltaX18(
    uint256 _principal,
    uint256 _liveDebt,
    uint256 _elapsed,
    uint256 _floor,
    uint256 _spread
)
    internal pure
    returns (uint256)
{
    if (_principal == 0 || _elapsed == 0) {
        return 0;
    }
    uint256 interestX18 = (_liveDebt > _principal ? _liveDebt - _principal :
    0) * WAD;
    uint256 owedX18 = Math.max(
        Math.mulDiv(_principal * _floor, _elapsed, YEAR),
        interestX18 + Math.mulDiv(_principal * _spread, _elapsed, YEAR)
    );
    return owedX18 - interestX18;
}
```

When a liquidation repays part of the debt, the corresponding collateral is seized from the borrower. The borrower therefore economically bears any interest included in that repayment. Nevertheless, if the liquidation reduces `_liveDebt` below `_principal`, the calculation sets `interestX18` to zero. The checkpoint consequently charges the full fee floor on the previous debt baseline, without crediting the borrower for the market interest economically borne through the

liquidation.

For example, consider a 1,000 RLUSD position with a 7% floor, a 1% spread, and a 6% Morpho rate. After one year, the debt reaches 1,060 RLUSD. If a partial liquidation repays 500 RLUSD and leaves 560 RLUSD outstanding, the borrower has economically borne 60 RLUSD of market interest. The next checkpoint observes debt below the previous 1,000 RLUSD baseline, treats the accrued interest as zero, and adds a 70 RLUSD Flare fee. If the interest had remained observable, the fee would have been 10 RLUSD. Excluding the liquidation incentive, the borrower consequently owes 130 RLUSD in interest and fees, equivalent to 13% of the initial debt, rather than the advertised 7% effective rate.

Impact

Borrowers liquidated between checkpoints can owe a higher Flare fee than the documented effective-rate formula implies. The potential overcharge scales with the time elapsed since the preceding checkpoint; therefore, regular checkpointing should keep the additional fee limited. However, the contracts do not enforce a maximum checkpoint interval, so larger discrepancies remain possible for dormant positions.

Recommendations

If detecting liquidations reliably on-chain is not practical, disclose this fee behavior in user-facing materials as an exception to the advertised effective-rate formula. Borrowers should also be informed that calling the permissionless `checkpointFee` function as their position approaches liquidation can reduce their exposure. More frequent checkpoints shorten the period whose accrued interest may be obscured by a subsequent liquidation.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit `bb8b7890` ↗.

Flare provided the following comment:

> Alongside the principal, the fee state now records its borrow shares and the collateral the position is expected to hold, in two appended storage fields. Every deposit and withdrawal through the account's own rails updates that expectation, so a shortfall against it identifies a liquidation. The checkpoint then values the recorded shares at the market's current totals to estimate the interest hidden by the liquidation. A repayment outside the rails alone remains uncredited. If a liquidation occurs in the same checkpoint period, before or after that repayment, the reconstructed interest credit also includes the repaid shares. Deposits with a recorded principal now checkpoint first, ending the credited period. The estimate's remaining limitations are documented in the specification under "Checkpoint accounting", and Flare's executor aims to checkpoint promptly after observing a liquidation.

3.3. A non-zero fxrpOft bridge fee would block cross-chain moves

Target	BridgingSubaccountBase, MorphoLoanSubaccountRemote		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `_bridgeToSelf` function in the BridgingSubaccountBase contract and the `_bridgeHome` function in the MorphoLoanSubaccountRemote contract both use LayerZero to send FXRP value to another chain:

```
SendParam memory sendParam = SendParam({
    // [...]
    amountLD: _amount,
    minAmountLD: _amount,
    // [...]
});
// [...]
(MessagingReceipt memory receipt, ) = IOFT(/* [...] */).send{value:
    fee.nativeFee}(
    sendParam,
    fee,
    // [...]
);
```

Both SendParam constructions specify zero slippage tolerance on the amount of the OFT token (FXRP) sent.

However, the FXRP OFT contracts can have fees set on them, because they inherit LayerZero fee modules (OFTFeeUpgradeable/OFTAdapterFeeUpgradeable) and expose `setDefaultFeeBps` and `setFeeBps`. If these fees are ever nonzero, then the above sends will fail, because the amount received will be below the set `minAmountLD`.

Impact

If the FXRP OFT contracts ever have a fee set on them, these cross-chain sends will fail.

Recommendations

We recommend setting an acceptable slippage tolerance. Alternatively, we recommend documenting the assumption that the FXRP OFT contracts will never have a fee set on them.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit 8b48c424.

Flare provided the following comment:

> Each token-bearing move now carries a signed minimum for what must arrive: minCollateralAmount on borrow, and minAmount on depositCollateral and on withdraw for the return leg. It is passed as minAmountLD, with amountLD the full amount sent. An adapter fee within the signed allowance goes through, and one above it reverts rather than coming out of the user's collateral. The withdraw action data gained a sixth field for the return leg's minimum. Before the fix, the issue could also only have arisen by Flare's own choice: Flare owns the FXRP OFT contracts, so only Flare could set a non-zero fee, and such a send would have failed closed without losing funds.

3.4. Accounts deployed through the singleton factory bypass pausing

Target	SubaccountsFacet		
Category	Business Logic	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `createSubaccount` function in the SubaccountsFacet has the `notPaused` modifier, which makes it revert if the contract is paused:

```
function createSubaccount(
    // [...]
)
    external
    notPaused
    returns (address)
{
    // [...]
    return Subaccounts.getOrCreateSubaccount(parentPersonalAccount,
    _subaccountType);
}
```

However, in the Subaccounts library, the `createSubaccount` function (which is called by `getOrCreateSubaccount`) calls into the singleton EIP-2470 factory to do the actual deployment:

```
function createSubaccount(
    // [...]
)
    internal
    returns (address _subaccount)
{
    // [...]
    // deploy via EIP-2470 singleton factory using CREATE2
    IISingletonFactory(PersonalAccounts.SINGLETON_FACTORY).deploy(bytecode,
    bytes32(0));
    // [...]
}
```

The proxy initializer does not call anything that checks whether the controller is paused.

Impact

The `notPaused` modifier implies that account creation does not work while the controller is paused. But because the EIP-2470 factory is permissionless, anyone can call it to deploy the contract even while the controller is paused.

This is also present in PersonalAccounts, but that was not changed in this diff.

Recommendations

If deployment should be blocked when the controller is paused, then we recommend having the initializer check that and refuse to proceed if it is paused.

Remediation

This issue has been acknowledged by Flare, and fixes were implemented in the following commits:

- 16d8e03b ↗
- 799990e0 ↗

3.5. ERC-20 approval is not zeroed in some cases

Target	BridgingSubaccountBase, MorphoLoanSubaccountRemote		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

When this project sets ERC-20 approval before an external contract call, even when that approval is expected to be consumed, it still zeroes the approval afterwards, just in case. However, in the reviewed changes, there were two locations where approvals were not zeroed afterwards.

In the BridgingSubaccountBase contract, in the `_sweepRedeem` function, the approval is not reset after the parent call:

```
fxrp.forceApprove(parent, _amount);
_redeemedAmount = IPersonalAccount(parent).redeemFromSubaccount{value:
    _fee}(_amount);
```

Secondly, in the MorphoLoanSubaccountRemote contract, in the `_supplyBalance` function, the approval is not reset after the call to Morpho:

```
fxrp.forceApprove(_market.morpho, _supplied);
IMorpho(_market.morpho).supplyCollateral(
    _market.marketParams, _supplied, address(this), ""
);
```

Impact

If the contracts do not consume all of the approval allotted to them, then there will be dangling approval after these two pieces of logic run.

Recommendations

We recommend setting the approval back to zero after these calls.

Remediation

This issue has been acknowledged by Flare, and a fix was implemented in commit c0a25c22 ↗.

4. Patch Review

The purpose of this section is to document the core changes in the codebase made within the commit range 91244b01 ↗ to 6905db84 ↗.

4.1. Notable changes

The following are notable changes made to the codebase.

- **Loan subaccounts.** `SubaccountsFacet`, `Subaccounts`, and `SubaccountProxy` introduce deterministic, type-specific staging accounts controlled by a parent `PersonalAccount`. Each account is deployed through the EIP-2470 singleton factory at an address derived from the diamond, parent account, and subaccount type. This derivation allows corresponding accounts to share the same address across supported chains.
- **Parent-authorized loan instructions.** The Flare-side `LoanSubaccount` exposes `borrow`, `depositCollateral`, `withdraw`, and `cancel`. These functions may be called only by the parent `PersonalAccount`. They construct and validate LayerZero compose payloads on-chain and bridge FXRP to the corresponding remote account. `BridgingSubaccountBase` also provides the permissionless `sweepRedeem` function, which transfers idle FXRP to the parent account and redeems it through FAssets.
- **Authenticated remote execution.** `ComposeReceiverAccountBase` centralizes LayerZero compose-message authentication and replay protection for remote accounts. It validates the configured endpoint, FXRP OFT, source endpoint ID, originating account, pinned peer, and recomputed packet GUID. The credited token amount is decoded from the authenticated OFT message rather than trusted from the compose payload.
- **Type-specific bridge routing.** `BridgeRoutesFacet` and `BridgeRoutes` configure the FXRP OFT adapter, pin a peer for each supported remote chain, and permanently associate each bridging subaccount type with a destination chain. `setSubaccountTypeRouteSuspended` provides a reversible suspension mechanism. Supporting another chain or Morpho market requires registering a distinct subaccount type.
- **Remote Personal Accounts.** `PersonalAccountRemote` represents users on the loan chain without requiring a local private key. `PersonalAccount.sendCrossChainUserOp` sends an XRPL-authorized batch to the corresponding remote address. `PersonalAccountRemote` executes the batch only after `ComposeReceiverAccountBase` authenticates the compose message.
- **Morpho loan positions.** `MorphoLoanSubaccountRemote` maintains a single Morpho position in the write-once market assigned to its subaccount type. It supports the composed `borrow`, `depositCollateral`, `withdraw`, and `cancel` actions, as well as the permissionless `repay`, `depositIdleFxrp`, and `checkpointFee` functions. The inherited `SubaccountBase` binds the account to its controller, parent Personal Account, and subaccount type. After `ComposeReceiverAccountBase` authenticates and decodes an incoming LayerZero message, the account dispatches each action to its corresponding internal handler. Applicable handlers record consumed operation IDs to prevent

duplicate execution.

- **Loan-fee accounting.** `MorphoLoanSubaccountRemote` and `LoanMath` calculate the loan fee using the greater of the configured floor rate or the Morpho market interest rate plus a configured spread. The remote account checkpoints accrued fees, allocates repayments between principal debt and fees, attempts to collect outstanding fees, and enforces collateral-coverage requirements before permitting additional borrowing or withdrawals.
- **Loan-chain diamond.** The remote chain uses a diamond deployed at the same address as its Flare counterpart, allowing corresponding Personal Accounts and loan subaccounts to resolve to identical addresses across chains. `PersonalAccountsFacet` and `SubaccountsFacet` make the diamond a type-aware beacon for remote account implementations. `ComposeTransportFacet` stores the authenticated LayerZero endpoint and pinned Flare origin, while `MorphoLoanMarketsFacet` manages write-once Morpho market assignments, borrow-suspension controls, and loan-fee terms. Although account addresses are identical across chains, the loan-chain diamond registers remote-specific implementations that authenticate compose messages and interact with the configured lending market.

4.2. Minor changes

The following are minor changes made to the codebase.

- Shared identity and initialization logic was extracted from `PersonalAccount` into `PersonalAccountBase`, which is inherited by both `PersonalAccount` and `PersonalAccountRemote`. The base contract centralizes one-time initialization and beacon implementation lookup while preserving the existing storage slots for `controllerAddress` and `xrplOwner`.
- `OwnableWithTimelock` now applies the timelock to ownership transfers, prevents ownership renunciation, and rejects native value attached to queued calls. `PauseFacet` also allows the owner to remove compromised unpausers immediately.
- Added `EthereumControllerInit` for the loan-chain diamond's initial facet cut. It registers support for the ERC-165, diamond-cut, diamond-loupe, and ERC-173 interfaces without applying the Flare-specific configuration performed by `MasterAccountControllerInit`.

5. Assessment Results

During our assessment on the scoped Loan Origination - Smart Accounts contracts, we discovered five findings. No critical issues were found. Three findings were of low impact and the remaining findings were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.